

Package ‘curvana’

March 9, 2026

Type Package

Title Tools to Analyze AFM Force–Distance Curves

Version 0.1.0

Description An all-in-one R package to read, manage, and analyze atomic force microscopy (AFM) force–distance curves, with support for parallel processing and user-supplied metadata.

License MIT + file LICENSE

Encoding UTF-8

URL <https://github.com/GaryZhangYue/fdafmR>

BugReports <https://github.com/GaryZhangYue/fdafmR/issues>

Imports circlize,
ComplexHeatmap,
future,
future.apply,
dplyr,
ggplot2,
ggrepel,
ggpubr,
methods,
tibble

RoxygenNote 7.3.2

Suggests knitr,
rmarkdown,
DT,
cowplot

VignetteBuilder knitr

Contents

analyze_a_curve_adhesive_force	2
analyze_a_curve_area	3
analyze_a_curve_interaction_distance	4
analyze_a_curve_noise	5
analyze_baseline	6
analyze_curves_adhesive_force	7

analyze_curves_all_analytical_metrics	8
analyze_curves_energy	9
analyze_curves_interaction_distance	11
analyze_curves_noise	12
analyze_sensitivity	13
area_trapezoid	14
area_triangle	14
calc_sensitivity	15
combineFdObj	16
createFdObjFromFolder	16
crossing_x0	17
extract	18
fdObj-class	18
find_baseline	19
plot_all_curve_metrics	20
plot_a_curve_metrics	22
plot_complex_heatmap	24
plot_deflection_curves	25
plot_fd_curves	26
plot_force_curve_energy	27
plot_metric_violin	28
plot_pca_biplot	30
show,fDObj-method	31
transform_a_curve	32
transform_curves	33

Index 34

analyze_a_curve_adhesive_force

Adhesive Force for a Single Transformed AFM Curve

Description

Computes the adhesive force as the most negative force value in a transformed curve and returns that value along with the separation distance at which it occurs. Note that the value will be negated. For example, if the most negative force is -2 nN, the function will return 2 nN as the adhesive force.

Usage

```
analyze_a_curve_adhesive_force(curve_df)
```

Arguments

curve_df A data.frame with columns:

- separation_distance_nm Numeric tip-sample separation (nm).
- force_nN Numeric force (nN).

Value

A named numeric vector of length 2:

adhesive_force_nN The most negative (minimum) force value in nN.

separation_distance_nm The separation distance (nm) at which the adhesive force occurs.

If no negative force exists (i.e., all forces are ≥ 0) or inputs are invalid, returns `c(adhesive_force_nN = NA_real_, separation_distance_nm = NA_real_)`.

Examples

```
df <- data.frame(
  separation_distance_nm = seq(-50, 200, by = 1),
  force_nN = 0.02 * seq(-50, 200, by = 1) + c(rep(0, 80), -3, rep(0, 170))
)
analyze_a_curve_adhesive_force(df)
```

analyze_a_curve_area	<i>Analyze the adhesive and repulsive areas of a single AFM curve (data.frame input)</i>
----------------------	--

Description

Calculates the total areas above (*repulsive*) and below (*adhesive*) the noise threshold for one curve supplied as a data.frame with columns separation_distance_nm (x) and force_nN (y).

Usage

```
analyze_a_curve_area(
  curve_df,
  noiseBand_low = -.Machine$double.eps,
  noiseBand_high = .Machine$double.eps
)
```

Arguments

curve_df	A data frame with at least two columns: separation_distance_nm Numeric x-coordinates (nm). Values < 0 are clamped to 0. force_nN Numeric y-coordinates (nN).
noiseBand_low	Numeric lower noise-band threshold (default: <code>-.Machine\$double.eps</code>). Adhesive area is counted where <code>force_nN < noiseBand_low</code> .
noiseBand_high	Numeric upper noise-band threshold (default: <code>.Machine\$double.eps</code>). Repulsive area is counted where <code>force_nN > noiseBand_high</code> .

Details

The computation proceeds in the given point order (no sorting). Segments entirely above the positive noise threshold or below the negative noise threshold are integrated as trapezoids, and segments that cross the noise thresholds are split at the interpolated crossing point and integrated as triangles. Areas within the noise band ($-\text{noise_cutoff}$ to $+\text{noise_cutoff}$) are ignored.

As a practical fix for small transformation offsets near the origin, any $\text{separation_distance_nm} < 0$ are clamped to 0 before integration. This works for both approach (x may run high $\rightarrow$ low) and retract (low $\rightarrow$ high) traces.

Value

A named numeric vector of length 2:

adhesive_area Total positive area where $y < \text{noiseBand_low}$.

repulsive_area Total positive area where $y > \text{noiseBand_high}$.

See Also

`area_trapezoid`, `area_triangle`, `crossing_x0`

Examples

```
df <- data.frame(
  separation_distance_nm = c(-0.02, 0, 0.10, 0.05, 0.20), # includes clamp and backtrack
  force_nN               = c( 0.50, 0, 0.30, -0.20, 0.10)
)
analyze_a_curve_area(df)
analyze_a_curve_area(df, noiseBand_low = -0.1, noiseBand_high = 0.1)
```

`analyze_a_curve_interaction_distance`

Analyze interaction distance from a single AFM curve (flexible thresholding)

Description

Detects the first significant deviation in force relative to a baseline window, marking rupture length (negative excursion) or repulsive distance (positive excursion). This function uses user-provided noise-band thresholds directly. Scanning can proceed from right-to-left (default) or left-to-right. When y-direction is negative, the function looks for the first point where $\text{force} < -\text{noise_cutoff}$ (i.e., the curve enters the adhesive region). When y-direction is positive, it looks for the last point where $\text{force} > \text{noise_cutoff}$ (i.e., the curve exits the repulsive region).

Usage

```
analyze_a_curve_interaction_distance(
  curve_df,
  baseline_span,
  y_direction = c("negative", "positive"),
  x_direction = c("left", "right"),
```

```

    noiseBand_low = NULL,
    noiseBand_high = NULL
)

```

Arguments

curve_df	data.frame with columns: - separation_distance_nm (numeric): x values (distance, nm) - force_nN (numeric): y values (force, nN)
baseline_span	Integer >= 1. Number of last points used as the baseline window.
y_direction	"negative" or "positive". - "negative": find first y < threshold (rupture-like; threshold is typically negative) - "positive": find first y > threshold (repulsion-like; threshold is typically positive)
x_direction	"left" or "right". - "left": scan from right to left (from just before baseline toward the origin) - "right": scan from left to right (from origin up to just before baseline)
noiseBand_low	Numeric scalar threshold for negative-direction detection. Required when y_direction = "negative".
noiseBand_high	Numeric scalar threshold for positive-direction detection. Required when y_direction = "positive".

Value

c(distance = x_at_first_excursion_or_NA, threshold = numeric_threshold_used)

analyze_a_curve_noise *Determine single-curve noise cutoff from baseline force noise*

Description

Computes a positive noise cutoff (nN) from the baseline window of one transformed AFM curve.

Usage

```

analyze_a_curve_noise(
  curve_df,
  baseline_span,
  threshold_method = c("sd", "mad", "quantile", "fixed"),
  multiplier = 3,
  mad_constant = 1.4826,
  quantile_low = 0.25,
  quantile_high = 0.75,
  fixed_low = NULL,
  fixed_high = NULL
)

```

Arguments

curve_df	data.frame with columns: - separation_distance_nm (numeric): x values (distance, nm) - force_nN (numeric): y values (force, nN)
baseline_span	Integer ≥ 1 . Number of last points used as the baseline window.
threshold_method	Character scalar controlling how baseline noise bands are estimated. One of <code>"sd"</code> , <code>"mad"</code> , <code>"quantile"</code> , <code>"fixed"</code> (default <code>"sd"</code>): <code>"sd"</code> : symmetric bands from baseline SD, i.e. $\pm \text{sd}(y_{\text{base}}) \times \text{multiplier}$. <code>"mad"</code> : symmetric robust bands from MAD, i.e. $\pm \text{mad}(y_{\text{base}}) \times \text{mad_constant} \times \text{multiplier}$. <code>"quantile"</code> : asymmetric empirical bands using <code>quantile_low</code> and <code>quantile_high</code> , each scaled by <code>multiplier</code> . <code>"fixed"</code> : user-specified <code>fixed_low</code> and <code>fixed_high</code> , each scaled by <code>multiplier</code> .
multiplier	Numeric ≥ 0 scaling multiplier for all methods. Default 3.
mad_constant	Scaling for MAD to be SD-equivalent. Default 1.4826.
quantile_low	Lower quantile probability for "quantile" method. Default 0.25.
quantile_high	Upper quantile probability for "quantile" method. Default 0.75.
fixed_low	Numeric lower band (nN) for "fixed" method. Default NULL.
fixed_high	Numeric upper band (nN) for "fixed" method. Default NULL.

Value

Named numeric vector: `c(noiseBand_low = value, noiseBand_high = value)`.

analyze_baseline	<i>Analyze Baseline from AFM Raw Curves in an fdObj</i>
------------------	---

Description

Identifies baseline segments in AFM raw force-distance curves using pre-calculated sensitivity values, and stores baseline values and segments in the `fdObj`.

Usage

```
analyze_baseline(
  fdObj,
  least_length = 150,
  useCurve = NULL,
  slp_threshold = 0.001,
  std_threshold = 0.005,
  threads = 1
)
```

Arguments

fdObj	An object of class fdObj.
least_length	Integer. Minimum number of points in the baseline segment.
useCurve	Character. Either "approach" or "retract" to specify which raw curve to use.
slp_threshold	Numeric. Maximum absolute slope allowed for baseline detection (default = 0.001).
std_threshold	Numeric. Maximum standard error of the slope (default = 0.005).
threads	Integer. Number of parallel threads to use (default = 1).

Value

An updated fdObj with baseline values in the metadata column baseline_V, the minimum number of points in the baseline segment baseline_span, and baseline segments in baseline_segment.

analyze_curves_adhesive_force

Adhesive Force for All Transformed Curves in an fdObj

Description

Applies analyze_a_curve_adhesive_force() to each transformed curve in an fdObj and stores results in metadata:

- adhesive_force_nN_<dir>
- adhesive_sep_nm_<dir>

Usage

```
analyze_curves_adhesive_force(fdObj, useCurve = "retract", threads = 1)
```

Arguments

fdObj	An object of class fdObj.
useCurve	Character. Either "approach" or "retract" (default "retract").
threads	Integer. Number of parallel workers (default 1).

Details

Where <dir> is "approach" or "retract".

Behavior:

- If a curve is present and valid but has no negative force, force = 0 and sep = NA.
- If a curve cannot be analyzed at all (missing/invalid transformed data), both are NA.

Value

The updated fdObj.

analyze_curves_all_analytical_metrics

Run full analytical metrics pipeline on transformed curves in an fdObj

Description

Wrapper around:

- analyze_curves_noise()
- analyze_curves_adhesive_force()
- analyze_curves_energy()
- analyze_curves_interaction_distance() (optional negative and/or positive runs)

Usage

```
analyze_curves_all_analytical_metrics(
  fdObj,
  useCurve = c("retract", "approach", "both"),
  threads = 1,
  noise_baseline_span = "automatic",
  noise_threshold_method = c("sd", "mad", "quantile", "fixed"),
  noise_multiplier = 3,
  noise_mad_constant = 1.4826,
  noise_quantile_low = 0.25,
  noise_quantile_high = 0.75,
  noise_fixed_low = NULL,
  noise_fixed_high = NULL,
  analyze_adhesive_force = TRUE,
  analyze_energy = TRUE,
  analyze_rupture_distance = TRUE,
  analyze_rupture_distance_baseline_span = "automatic",
  analyze_rupture_distance_x_direction = c("left", "right"),
  analyze_repulsive_distance = TRUE,
  analyze_repulsive_distance_baseline_span = "automatic",
  analyze_repulsive_distance_x_direction = c("right", "left")
)
```

Arguments

fdObj	An object of class fdObj.
useCurve	Character; one of c("retract", "approach", "both"). Default "both".
threads	Integer. Number of parallel workers passed to wrapped functions (default 1).
noise_baseline_span	Either a single integer ≥ 1 , or "automatic". Passed to analyze_curves_noise().
noise_threshold_method	Character scalar; one of c("sd", "mad", "quantile", "fixed"). Passed to analyze_curves_noise().
noise_multiplier	Numeric ≥ 0 . Passed to analyze_curves_noise().

noise_mad_constant
Numeric. Passed to analyze_curves_noise().

noise_quantile_low
Numeric in [0,1]. Passed to analyze_curves_noise().

noise_quantile_high
Numeric in [0,1]. Passed to analyze_curves_noise().

noise_fixed_low
Numeric or NULL. Passed to analyze_curves_noise() when method is "fixed".

noise_fixed_high
Numeric or NULL. Passed to analyze_curves_noise() when method is "fixed".

analyze_adhesive_force
Logical. If TRUE, runs analyze_curves_adhesive_force(). Default TRUE.

analyze_energy Logical. If TRUE, runs analyze_curves_energy(). Default TRUE.

analyze_rupture_distance
Logical. If TRUE, runs interaction-distance analysis with y_direction = "negative". Default TRUE.

analyze_rupture_distance_baseline_span
Either a single integer ≥ 1 , or "automatic". Passed to negative-direction analyze_curves_interaction_distance().

analyze_rupture_distance_x_direction
Character; one of c("left", "right"). Default "left". Passed to negative-direction analyze_curves_interaction_distance().

analyze_repulsive_distance
Logical. If TRUE, runs interaction-distance analysis with y_direction = "positive". Default TRUE.

analyze_repulsive_distance_baseline_span
Either a single integer ≥ 1 , or "automatic". Passed to positive-direction analyze_curves_interaction_distance().

analyze_repulsive_distance_x_direction
Character; one of c("right", "left"). Default "right". Passed to positive-direction analyze_curves_interaction_distance().

Details

This function executes the "Analytical metrics calculation" steps in one call. Users can run for approach curves, retract curves, or both.

Value

Updated fdObj with analytical metrics written to metadata.

analyze_curves_energy *Adhesive and Repulsive Energy (aJ) for All Transformed Curves in an fdObj*

Description

Applies `analyze_a_curve_area()` to each transformed curve in an `fdObj` and stores the resulting energies (areas under the force–distance curve) in the metadata as:

- `adhesive_energy_aJ_<dir>` — total area where $y < \text{noiseBand_low}$
- `repulsive_energy_aJ_<dir>` — total area where $y > \text{noiseBand_high}$

where `<dir>` is "approach" or "retract".

Usage

```
analyze_curves_energy(fdObj, useCurve = c("retract", "approach"), threads = 1)
```

Arguments

<code>fdObj</code>	An object of class <code>fdObj</code> .
<code>useCurve</code>	Character; must be one of <code>c("retract", "approach")</code> .
<code>threads</code>	Integer. Number of parallel workers (default 1). Uses future+future.apply when <code>threads > 1</code> .

Details

Energies are reported in **attojoules (aJ)**, since $1 \text{ nNm} = 10^{-18} \text{ J} = 1 \text{ aJ}$.

The function preserves the point order (no sorting), tolerates backtracking in `x`, and inherits clamping of $x < 0$ to 0 from `analyze_a_curve_area()`.

Behavior:

- If a curve is present/valid but yields no finite result, both energies are NA.
- If no transformed curves exist for the selected direction, both metadata columns are created and filled with NA.

This function assumes precomputed noise-band columns already exist in `fdObj@metadata` and uses them directly:

- `noiseBand_low_nN_<useCurve>`
- `noiseBand_high_nN_<useCurve>`

Example for `useCurve = "retract"`: `noiseBand_low_nN_retract` and `noiseBand_high_nN_retract`.

Value

The updated `fdObj` with two new metadata columns: `adhesive_energy_aJ_<dir>` and `repulsive_energy_aJ_<dir>`.

See Also

`analyze_a_curve_area`, `analyze_curves_adhesive_force`

analyze_curves_interaction_distance

Interaction Distance (nm) for All Transformed Curves in an fdObj

Description

Applies `analyze_a_curve_interaction_distance()` to each transformed curve in an `fdObj` and stores the resulting interaction distances and thresholds in the metadata. These measure the first significant excursion in force relative to a baseline, either repulsive (positive force deviation) or rupture (negative).

Usage

```
analyze_curves_interaction_distance(
  fdObj,
  useCurve = c("retract", "approach"),
  threads = 1,
  baseline_span,
  y_direction = c("negative", "positive"),
  x_direction = c("left", "right")
)
```

Arguments

<code>fdObj</code>	An object of class <code>fdObj</code> .
<code>useCurve</code>	Character; must be one of <code>c("retract", "approach")</code> .
<code>threads</code>	Integer. Number of parallel workers (default 1).
<code>baseline_span</code>	Either a single integer ≥ 1 , or the string "automatic". When "automatic", per-curve baseline spans are read from <code>fdObj@metadata\$baseline_span_<useCurve></code> .
<code>y_direction</code>	Detection mode: "negative": detects first point with <code>force_nN < noiseBand_low</code> (curve enters adhesive region; threshold is typically negative). "positive": detects last point with <code>force_nN > noiseBand_high</code> (curve exits repulsive excursion; threshold is typically positive).
<code>x_direction</code>	Scan direction for each curve: "left": right-to-left, from just before baseline toward origin. "right": left-to-right, from origin toward just before baseline.

Details

This function assumes precomputed noise-band columns already exist in `fdObj@metadata` and uses them directly:

- if `y_direction = "negative"`: `noiseBand_low_nN_<useCurve>`
- if `y_direction = "positive"`: `noiseBand_high_nN_<useCurve>`

Example for `useCurve = "retract"`: `noiseBand_low_nN_retract` or `noiseBand_high_nN_retract`.

Value

The updated fdObj with two new metadata columns: <type>_distance_nm_<dir> and <type>_threshold_nN_<dir>, where <type> is "repulsive" or "rupture", depending on y_direction, and <dir> is "approach" or "retract".

See Also

analyze_a_curve_interaction_distance

analyze_curves_noise *Noise-band Analysis (nN) for All Transformed Curves in an fdObj*

Description

Applies analyze_a_curve_noise() to each transformed curve in an fdObj and stores per-curve lower/upper noise bands in metadata:

- noiseBand_low_nN_<dir>
- noiseBand_high_nN_<dir>

where <dir> is "approach" or "retract".

Usage

```
analyze_curves_noise(
  fdObj,
  useCurve = c("retract", "approach"),
  threads = 1,
  baseline_span,
  threshold_method = c("sd", "mad", "quantile", "fixed"),
  multiplier = 3,
  mad_constant = 1.4826,
  quantile_low = 0.25,
  quantile_high = 0.75,
  fixed_low = NULL,
  fixed_high = NULL
)
```

Arguments

fdObj	An object of class fdObj.
useCurve	Character; must be one of c("retract", "approach").
threads	Integer. Number of parallel workers (default 1).
baseline_span	Either a single integer >= 1, or the string "automatic". When "automatic", per-curve baseline spans are read from fdObj@metadata\$baseline_span_<useCurve>.
threshold_method	Character scalar controlling how baseline noise bands are estimated. One of c("sd", "mad", "quantile", "fixed") (default "sd").
multiplier	Numeric >= 0 scaling multiplier for all methods. Default 3.

mad_constant	Scaling for MAD to be SD-equivalent. Default 1.4826.
quantile_low	Lower quantile probability for "quantile" method. Default 0.25.
quantile_high	Upper quantile probability for "quantile" method. Default 0.75.
fixed_low	Numeric lower band (nN) for "fixed" method. Default NULL.
fixed_high	Numeric upper band (nN) for "fixed" method. Default NULL.

Value

The updated fdObj with two new metadata columns: noiseBand_low_nN_<dir> and noiseBand_high_nN_<dir>.

See Also

analyze_a_curve_noise

analyze_sensitivity	<i>Add Sensitivity Estimates to an fdObj</i>
---------------------	--

Description

Applies sensitivity estimation to each rawCurve in an fdObj and stores the sensitivity values in the metadata and the segment used in the senscal_segment slot.

Usage

```
analyze_sensitivity(
  fdObj,
  end = 200,
  intv = 4,
  useCurve = "approach",
  threads = 1
)
```

Arguments

fdObj	An object of class fdObj.
end	Integer. The maximum index in raw curves to consider (e.g., 200).
intv	Integer. Chunk size for sensitivity calculation (e.g., 4).
useCurve	Character. Either "approach" or "retract" to determine which curve to use.
threads	Number of parallel threads to use (default = 1).

Value

An updated fdObj with sensitivity values in metadata and segments in senscal_segment.

area_trapezoid	<i>Calculate the trapezoid area between two points</i>
----------------	--

Description

Computes the absolute area of a trapezoid formed by two points (x1, y1) and (x2, y2) along the x-axis. This is used when both points are on the same side of the x-axis (either both positive or both negative).

Usage

```
area_trapezoid(x1, y1, x2, y2)
```

Arguments

x1	Numeric. The x-coordinate of the first point.
y1	Numeric. The y-coordinate of the first point.
x2	Numeric. The x-coordinate of the second point.
y2	Numeric. The y-coordinate of the second point.

Value

A single numeric value representing the positive trapezoid area between the two points.

See Also

[area_triangle()], [crossing_x0()], [calculate_area()]

Examples

```
area_trapezoid(0, 2, 1, 4) # 0.5 * (1 - 0) * (2 + 4) = 3
area_trapezoid(0, -2, 1, -4) # same absolute area = 3
```

area_triangle	<i>Calculate the triangle area between a curve point and the x-axis</i>
---------------	---

Description

Computes the absolute area of a triangle formed by a point (x1, y1) on the curve and its intersection with the x-axis at (x0, 0). This function assumes one vertex lies on the x-axis (y = 0).

Usage

```
area_triangle(x1, y1, x0)
```

Arguments

x1	Numeric. The x-coordinate of the curve point.
y1	Numeric. The y-coordinate of the curve point.
x0	Numeric. The x-coordinate of the x-axis intersection point (where $y = 0$).

Value

A single numeric value representing the positive triangle area.

See Also

[area_trapezoid()], [crossing_x0()], [calculate_area()]

Examples

```
# Triangle between (0, 4) and (1, 0)
area_triangle(0, 4, 1) # 0.5 * |1 - 0| * |4| = 2
```

calc_sensitivity	<i>Calculate Sensitivity from a Segment of AFM Data</i>
------------------	---

Description

This function identifies a linear segment from AFM deflection vs. piezo extension data and calculates the sensitivity (slope) using linear regression. It iteratively adds chunks of data to build a segment with high linearity ($R^2 > 0.999$) and optionally removes outliers to maintain this criterion.

Usage

```
calc_sensitivity(end, intv = 4, x, y)
```

Arguments

end	Integer. The maximum index to consider in the 'x' and 'y' vectors (i.e., up to which point to search).
intv	Integer. The chunk size to use when iteratively selecting data segments.
x	Numeric vector. The piezo extension values (e.g., distance in nm).
y	Numeric vector. The deflection values (e.g., voltage).

Value

A list of three elements:

senscal_segment a dataframe of a subset of points in the original raw curve which are used for sensitivity calibration

sensitivity The calculated sensitivity (slope) as a numeric value, or NULL if not computable.

Examples

```
x <- seq(0, 100, by = 1)
y <- 0.02 * x + rnorm(length(x), sd = 0.01)
calc_sensitivity(end = 80, intv = 10, x = x, y = y)
```

combineFdObj	<i>Combine two fdObj objects</i>
--------------	----------------------------------

Description

Combine two fdObj objects

Usage

```
combineFdObj(x, y)
```

Arguments

x	A 'fdObj' object
y	A 'fdObj' object. This object cannot have any overlapping sample names with the first fdObj.

Value

A new combined 'fdObj' object. metadata matrix will be concatenated using bind_rows (dplyr). All slots will be concatenated.

createFdObjFromFolder	<i>Constructor for fdObj from a folder of AFM force–distance curve files</i>
-----------------------	--

Description

Constructor for fdObj from a folder of AFM force–distance curve files

Usage

```
createFdObjFromFolder(
  folder,
  suffix = ".txt",
  pattern = "",
  Calc_Ramp_Ex_nm = "Calc_Ramp_Ex_nm",
  Calc_Ramp_Rt_nm = "Calc_Ramp_Rt_nm",
  Defl_V_Ex = "Defl_V_Ex",
  Defl_V_Rt = "Defl_V_Rt",
  metadata = NULL,
  threads = 1
)
```


Arguments

folder	Path to a folder containing the raw AFM curve files
suffix	File extension to look for (e.g., ".txt")
pattern	Optional string pattern to filter files (e.g., "experiment"); default is "" (no filtering)
Calc_Ramp_Ex_nm	Column name for approach distance (default: "Calc_Ramp_Ex_nm")
Calc_Ramp_Rt_nm	Column name for retract distance (default: "Calc_Ramp_Rt_nm")
Defl_V_Ex	Column name for approach deflection (default: "Defl_V_Ex")
Defl_V_Rt	Column name for retract deflection (default: "Defl_V_Rt")
metadata	Optional data.frame. If provided, file names (excluding the suffix) matched to rownames(metadata) will be read in. Returns error if any files not found.
threads	Number of parallel threads to use for file reading. Default = 1 (sequential).

Value

An object of class fdObj

crossing_x0	<i>Interpolate the x-intercept (x0) where $y = 0$</i>
-------------	--

Description

Performs linear interpolation between two points (x1, y1) and (x2, y2) to find the x-coordinate (x0) where the line crosses the x-axis ($y = 0$). Only valid when y1 and y2 have opposite signs.

Usage

```
crossing_x0(x1, y1, x2, y2)
```

Arguments

x1	Numeric. The x-coordinate of the first point.
y1	Numeric. The y-coordinate of the first point.
x2	Numeric. The x-coordinate of the second point.
y2	Numeric. The y-coordinate of the second point.

Value

A single numeric value for the interpolated x0 coordinate. Returns NA if no valid sign change exists between y1 and y2.

See Also

[area_trapezoid()], [area_triangle()], [calculate_area()]

Examples

```
crossing_x0(0, 4, 2, -2) # 1.333 - line crosses y=0 between x=0 and x=2
crossing_x0(0, 4, 2, 6)  # NA - no sign change
```

extract

Extract subset of fdObj by sample or metadata column

Description

Extract subset of fdObj by sample or metadata column

Usage

```
extract(fdObj, by_sample = NULL, by_col = NULL)
```

Arguments

fdObj	An object of class fdObj
by_sample	A character vector of sample names to keep
by_col	A named list like list(colname = value) to filter metadata (e.g., list(type = "control"))

Value

A subsetting fdObj object

fdObj-class

An S4 class to hold AFM force-distance raw and processed curves with metadata

Description

An S4 class to hold AFM force-distance raw and processed curves with metadata

Slots

rawCurves A named list of data.frames, each with 4 columns (raw AFM data)

approachCurves A named list of 2-column data.frames (distance vs force from approach segment)

retractCurves A named list of 2-column data.frames (distance vs force from retract segment)

metadata A data.frame with row names matching names(rawCurves). Stores metadata and calculated features.

senscal_segment A list with two slots: approach and retract, each is a named list of data.frames, storing the part of the raw curve used for sensitivity calculation.

baseline_segment A list with two slots: approach and retract, each is a named list of data.frames, storing the part of the raw curve used for baseline calculation

find_baseline

Identify Baseline Segment in AFM Raw Force-Distance Curve

Description

This function identifies a flat, noise-stable baseline segment at the end of an AFM (Atomic Force Microscopy) raw force-distance curve. It applies a linear regression on a trailing window to determine if the segment is sufficiently flat and stable to be considered a baseline.

Usage

```
find_baseline(
  x,
  y,
  least_length,
  sensitivity,
  slp_threshold = 0.001,
  std_threshold = 0.005
)
```

Arguments

<code>x</code>	Numeric vector. Distance values (e.g., piezo positions or z-sensor values).
<code>y</code>	Numeric vector. Deflection values (e.g., in volts).
<code>least_length</code>	Integer. Minimum number of points in the baseline segment.
<code>sensitivity</code>	Numeric. Scaling factor for the deflection signal (e.g., detector sensitivity in V/nm or V/nN).
<code>slp_threshold</code>	Numeric. Maximum absolute slope for the segment to be considered flat (default: 0.001).
<code>std_threshold</code>	Numeric. Maximum standard error of the slope (default: 0.005).

Details

The function assumes the baseline occurs toward the tail end of the curve (typically during the retract phase). It is often used prior to baseline correction or contact point determination in AFM force curve analysis.

Value

A list with two elements:

baseline Numeric. The estimated baseline deflection value (in original units), or NULL if no valid segment is found.

segment Data frame with `x` and `y` columns representing the detected baseline segment, or NULL if not found.

Examples

```
# Simulated flat tail data
x <- 1:1000
y <- c(rnorm(980, 0.1, 0.05), rnorm(20, 0, 0.002)) # flat tail
result <- find_baseline(least_length = 15, sensitivity = 1, x = x, y = y)
print(result$baseline)
plot(x, y, type = "l"); lines(result$segment, col = "red", lwd = 2)
```

```
plot_all_curve_metrics
```

Plot and save curve metrics for all curves in an fdObj

Description

Iteratively runs `plot_a_curve_metrics()` over all entries in the selected transformed-curve slot (approach or retract) of an `fdObj`, and saves each plot to disk.

Usage

```
plot_all_curve_metrics(
  fdobj,
  useCurve = c("retract", "approach"),
  threads = 1,
  plot_raw = FALSE,
  annotate_noiseBand = TRUE,
  annotate_repulsive_energy = TRUE,
  annotate_adhesive_energy = TRUE,
  annotate_adhesive_force = TRUE,
  annotate_rupture_distance = TRUE,
  annotate_repulsive_distance = TRUE,
  xlim = NULL,
  ylim = NULL,
  title = NULL,
  base_size = 14,
  line_color = "grey35",
  point_color = "grey35",
  line_size = 0.7,
  point_size = 1.8,
  line_alpha = 0.9,
  point_alpha = 0.8,
  annotation_text_size = 3.3,
  interaction_label_color_rupture = "purple4",
  interaction_label_color_repulsive = "darkorange3",
  interaction_label_fill = "white",
  destination_folder = NULL,
  width = 12,
  height = 6,
  format = c("png", "pdf", "jpeg", "tiff", "bmp", "svg"),
  dpi = 300,
  units = c("in", "cm", "mm"),
```

```
    ...  
  )
```

Arguments

fdobj	An object of class fdObj.
useCurve	Character. Either "retract" or "approach".
threads	Integer. Number of parallel workers (default 1). Uses future+future.apply when threads > 1.
plot_raw	Logical. Passed to plot_a_curve_metrics().
annotate_noiseBand	Logical. Passed to plot_a_curve_metrics().
annotate_repulsive_energy	Logical. Passed to plot_a_curve_metrics().
annotate_adhesive_energy	Logical. Passed to plot_a_curve_metrics().
annotate_adhesive_force	Logical. Passed to plot_a_curve_metrics().
annotate_rupture_distance	Logical. Passed to plot_a_curve_metrics().
annotate_repulsive_distance	Logical. Passed to plot_a_curve_metrics().
xlim	Optional numeric vector of length 2 for x-axis limits.
ylim	Optional numeric vector of length 2 for y-axis limits.
title	Character. Plot title passed to plot_a_curve_metrics().
base_size	Numeric. Base theme text size.
line_color	Character. Line color for curves.
point_color	Character. Point color for curves.
line_size	Numeric. Line width for curves.
point_size	Numeric. Point size for curves.
line_alpha	Numeric. Line alpha for curves.
point_alpha	Numeric. Point alpha for curves.
annotation_text_size	Numeric. Text size for annotation labels.
interaction_label_color_rupture	Character. Label color for rupture annotations.
interaction_label_color_repulsive	Character. Label color for repulsive annotations.
interaction_label_fill	Character. Fill color for interaction labels.
destination_folder	Character. Output folder path. If NULL, a timestamped folder is created in the current working directory.
width	Numeric. Plot width passed to ggplot2::ggsave().
height	Numeric. Plot height passed to ggplot2::ggsave().

format	Character. Output graphics format. One of "png", "pdf", "jpeg", "tiff", "bmp", "svg".
dpi	Numeric. Resolution passed to ggplot2::ggsave().
units	Character. Units for width/height. One of "in", "cm", "mm".
...	Additional ggplot2 components passed to plot_a_curve_metrics().

Value

A data.frame with one row per curve and columns: curve_name, file_path, status, and error_message. The output folder is attached as attr(result, "destination_folder").

plot_a_curve_metrics *Plot one transformed curve with optional metadata annotations*

Description

High-level visualization helper that takes an fdObj and one curve name, plots the transformed force-distance curve, and optionally overlays selected annotations from predefined metadata columns for the selected useCurve.

Usage

```
plot_a_curve_metrics(
  fdobj,
  curve_name,
  useCurve = c("retract", "approach"),
  plot_raw = FALSE,
  annotate_noiseBand = TRUE,
  annotate_repulsive_energy = TRUE,
  annotate_adhesive_energy = TRUE,
  annotate_adhesive_force = TRUE,
  annotate_rupture_distance = TRUE,
  annotate_repulsive_distance = TRUE,
  xlim = NULL,
  ylim = NULL,
  title = NULL,
  base_size = 14,
  line_color = "grey35",
  point_color = "grey35",
  line_size = 0.7,
  point_size = 1.8,
  line_alpha = 0.9,
  point_alpha = 0.8,
  annotation_text_size = 3.3,
  interaction_label_color_rupture = "purple4",
  interaction_label_color_repulsive = "darkorange3",
  interaction_label_fill = "white",
  ...
)
```

Arguments

<code>fdobj</code>	An object of class <code>fdObj</code> .
<code>curve_name</code>	Character. Name of one curve to plot. Must match <code>rownames(fdobj@metadata)</code> .
<code>useCurve</code>	Character. Either "retract" or "approach".
<code>plot_raw</code>	Logical. If TRUE, also plot the corresponding raw curve and return a two-column panel using <code>cowplot::plot_grid()</code> .
<code>annotate_noiseBand</code>	Logical. If TRUE, annotate noise band from <code>noiseBand_low_nN_<useCurve></code> and <code>noiseBand_high_nN_<useCurve></code> , and add "Noise band" labels above and below the band boundaries.
<code>annotate_repulsive_energy</code>	Logical. If TRUE, add repulsive energy value to subtitle when available.
<code>annotate_adhesive_energy</code>	Logical. If TRUE, add adhesive force and adhesive energy values to subtitle when available.
<code>annotate_adhesive_force</code>	Logical. If TRUE, annotate adhesive force point and separation using arrow + label.
<code>annotate_rupture_distance</code>	Logical. If TRUE, annotate rupture distance/threshold using arrow + label.
<code>annotate_repulsive_distance</code>	Logical. If TRUE, annotate repulsive distance/threshold using arrow + label.
<code>xlim</code>	Optional numeric vector of length 2 for x-axis limits.
<code>ylim</code>	Optional numeric vector of length 2 for y-axis limits.
<code>title</code>	Character. Plot title.
<code>base_size</code>	Numeric. Base theme text size.
<code>line_color</code>	Character. Line color for curves.
<code>point_color</code>	Character. Point color for curves.
<code>line_size</code>	Numeric. Line width for curves.
<code>point_size</code>	Numeric. Point size for curves.
<code>line_alpha</code>	Numeric. Line alpha for curves.
<code>point_alpha</code>	Numeric. Point alpha for curves.
<code>annotation_text_size</code>	Numeric. Text size used for annotation labels.
<code>interaction_label_color_rupture</code>	Character. Text/line color for rupture annotation label.
<code>interaction_label_color_repulsive</code>	Character. Text/line color for repulsive annotation label.
<code>interaction_label_fill</code>	Character. Fill color for interaction label box.
<code>...</code>	Additional <code>ggplot2</code> components to add to the transformed-curve plot (e.g., <code>theme()</code> , <code>scale_*()</code> , <code>coord_*()</code> , <code>labs()</code> , or extra <code>geom_*()</code> layers).

Value

A `ggplot2` object, or a `cowplot` grid object when `plot_raw = TRUE`.

plot_complex_heatmap *Plot a scaled feature heatmap using ComplexHeatmap*

Description

Creates a heatmap where selected feature columns are shown as rows and samples are shown as columns. Feature values are z-score scaled by row (feature) before plotting. Optionally annotates sample columns with up to two metadata columns.

Usage

```
plot_complex_heatmap(
  df,
  include_columns,
  annotate_columns = NULL,
  annotation_colors = NULL,
  cluster_rows = TRUE,
  cluster_columns = TRUE,
  show_row_dend = TRUE,
  show_column_dend = TRUE,
  show_row_names = TRUE,
  show_column_names = TRUE,
  heatmap_name = "z-score",
  row_title = "Features",
  column_title = NULL,
  draw = TRUE
)
```

Arguments

df	A data.frame containing feature and metadata columns. Each row is one sample.
include_columns	Character vector of feature column names to include as heatmap rows.
annotate_columns	Optional character vector (length 0 to 2) of metadata column names used to annotate heatmap columns (samples).
annotation_colors	Optional named list of color mappings for annotation columns passed to ComplexHeatmap::Heatmap(...).
cluster_rows	Logical; whether to cluster heatmap rows.
cluster_columns	Logical; whether to cluster heatmap columns.
show_row_dend	Logical; whether to display the row dendrogram.
show_column_dend	Logical; whether to display the column dendrogram.
show_row_names	Logical; whether to display row names (feature names).
show_column_names	Logical; whether to display column names (sample names).
heatmap_name	Character legend title for the scaled values.

row_title	Optional row title.
column_title	Optional column title.
draw	Logical; if TRUE, draws the heatmap immediately.

Value

A ComplexHeatmap object. When draw = TRUE, returns the object produced by ComplexHeatmap::draw().

plot_deflection_curves

Plot raw deflection vs piezo distance from rawCurves slot of fdObj

Description

Plot raw deflection vs piezo distance from rawCurves slot of fdObj

Usage

```
plot_deflection_curves(
  fdobj,
  curve = c("both", "approach", "retract"),
  group_curves_by = NULL,
  split_curves_by = NULL,
  point_size = 0.5,
  alpha = 0.5,
  line_alpha = 0.5,
  color_map = NULL
)
```

Arguments

fdobj	A 'fdObj' object
curve	Which segment(s) to plot: "approach", "retract", or "both"
group_curves_by	Metadata column to color by (optional)
split_curves_by	Metadata column to facet by (optional)
point_size	Size of points (default = 0.5)
alpha	Transparency of points (default = 0.6)
line_alpha	Transparency of connecting paths (default = 0.5)
color_map	Named vector of colors (optional)

Value

A ggplot2 object of the scatter plots showing raw deflection signal.

plot_fd_curves

*Plot Force-Distance Curves from an fdObj***Description**

Generates force-distance plots from the calibrated approach or retract curves in an fdObj.

Usage

```
plot_fd_curves(
  fdobj,
  curve = "both",
  group_curves_by = NULL,
  split_curves_by = NULL,
  color_map = NULL,
  point_size = 0.5,
  point_alpha = 0.6,
  line_alpha = 0.5,
  xlim = NULL,
  ylim = NULL,
  ...
)
```

Arguments

fdobj	An object of class fdObj.
curve	Character. Either "approach", "retract", or "both" (default = "both").
group_curves_by	Metadata column to color by (optional).
split_curves_by	Metadata column to facet by (optional).
color_map	Named vector of colors (optional).
point_size	Size of points (default = 0.5).
point_alpha	Transparency of points (default = 0.6).
line_alpha	Transparency of curve lines (default = 0.5).
xlim	Optional numeric vector of length 2 for x-axis limits.
ylim	Optional numeric vector of length 2 for y-axis limits.
...	Additional ggplot2 layers/settings to add to the plot (e.g., theme(...), scale_*(), labs(...)).

Value

A ggplot2 object showing the force-distance relationships from AFM curves.

plot_force_curve_energy

Plot a transformed force curve with energy regions highlighted

Description

Creates a ggplot visualization of a force-distance curve with colored regions indicating adhesive (blue) and repulsive (coral) energy areas based on a noise cutoff threshold. Internally calls `analyze_a_curve_area()` to calculate the energy values displayed in the plot subtitle.

Usage

```
plot_force_curve_energy(
  curve_df,
  noiseBand_low = -0.5,
  noiseBand_high = 0.5,
  title = "Interaction Energy Calculation with Noise Threshold",
  base_size = 14,
  point_size = 2,
  alpha_ribbon = 0.3,
  show_legend = TRUE
)
```

Arguments

<code>curve_df</code>	A data frame containing the transformed force curve with columns <code>separation_distance_nm</code> and <code>force_nN</code> .
<code>noiseBand_low</code>	Numeric lower noise-band threshold (nN). Adhesive region is where <code>force_nN < noiseBand_low</code> .
<code>noiseBand_high</code>	Numeric upper noise-band threshold (nN). Repulsive region is where <code>force_nN > noiseBand_high</code> .
<code>title</code>	Character. Title for the plot. Default is "Interaction Energy Calculation with Noise Threshold".
<code>base_size</code>	Numeric. Base font size for the plot. Default is 14.
<code>point_size</code>	Numeric. Size of points on the curve. Default is 2.
<code>alpha_ribbon</code>	Numeric. Transparency of the colored ribbon areas (0-1). Default is 0.3.
<code>show_legend</code>	Logical. Whether to show the legend. Default is TRUE.

Details

The function:

1. Calls `analyze_a_curve_area()` with `noiseBand_low` and `noiseBand_high`
2. Creates a region classification (Baseline, Adhesive, Repulsive)
3. Visualizes the noise band as a grey region
4. Colors adhesive regions (`force < noiseBand_low`) in steelblue
5. Colors repulsive regions (`force > noiseBand_high`) in coral
6. Displays calculated energy values in the plot subtitle

Value

A ggplot2 object showing the force curve with colored regions and energy values in the subtitle.

See Also

[analyze_a_curve_area()], [transform_a_curve()]

Examples

```
## Not run:
# Load example data
data(curvana_sample)
curve <- curvana_sample@retractCurves[[1]]
curve_transformed <- transform_a_curve(curve)

# Plot with default noise cutoff (0.5 nN)
plot_force_curve_energy(curve_transformed, noiseBand_low = -0.5, noiseBand_high = 0.5)

# Plot with larger noise band
plot_force_curve_energy(curve_transformed, noiseBand_low = -5, noiseBand_high = 5)

# Customize appearance
plot_force_curve_energy(curve_transformed, noiseBand_low = -1.0, noiseBand_high = 1.0,
                        title = "My Custom Title", base_size = 12)

## End(Not run)
```

plot_metric_violin	<i>Plot one metric as a violin plot</i>
--------------------	---

Description

Visualizes one metric column from a data frame using violin plots, optional jittered points, and an optional linear trend line. Faceting supports up to two columns. Supports one global test (ANOVA or Kruskal-Wallis) shown in subtitle, and pairwise tests (t-test or Wilcoxon) shown as significance asterisks.

Usage

```
plot_metric_violin(
  df,
  metric_name,
  group_by,
  color_by = NULL,
  color_map = NULL,
  facet_by = NULL,
  global_test = c("none", "anova", "kruskal"),
  pairwise_test = c("none", "t.test", "wilcox"),
  pairwise_comparisons = NULL,
  p_adjust_method = "BH",
  log10 = FALSE,
```

```

    add_points = TRUE,
    add_smooth = FALSE,
    point_alpha = 0.6,
    point_color = "black",
    point_size = 1.2,
    jitter_width = 0.15,
    violin_fill = "grey85",
    violin_color = "grey40",
    violin_alpha = 0.8,
    smooth_color = "steelblue",
    base_size = 10
  )

```

Arguments

<code>df</code>	A data.frame containing grouping and metric columns.
<code>metric_name</code>	Character scalar. One column name in <code>df</code> to plot on the y-axis.
<code>group_by</code>	Character scalar. Column name in <code>df</code> used for the x-axis grouping.
<code>color_by</code>	Character scalar. Column name in <code>df</code> used to color violins and points. Defaults to <code>group_by</code> when NULL.
<code>color_map</code>	Optional named character vector of colors for groups in <code>color_by</code> (e.g., <code>c(A = "#1b9e77", B = "#d95f02")</code>).
<code>facet_by</code>	Character vector of length 0 to 2 specifying facet columns. Use NULL for no faceting.
<code>global_test</code>	Character scalar. One of "none", "anova", or "kruskal". When not "none", the global test p-value is added as subtitle.
<code>pairwise_test</code>	Character scalar. One of "none", "t.test", or "wilcox". Pairwise comparisons are shown as significance asterisks.
<code>pairwise_comparisons</code>	Optional list of group-name pairs (e.g., <code>list(c("A", "B"), c("A", "C"))</code>). If NULL, all pairwise combinations are used.
<code>p_adjust_method</code>	Character p-value adjustment method passed to <code>ggpubr::stat_compare_means()</code> .
<code>log10</code>	Logical. If TRUE, applies log10 transform to the metric values before plotting and testing. Non-positive values are removed.
<code>add_points</code>	Logical. If TRUE, overlays jittered points.
<code>add_smooth</code>	Logical. If TRUE, overlays <code>geom_smooth(method = "lm")</code> within each facet.
<code>point_alpha</code>	Numeric point transparency.
<code>point_color</code>	Character point color.
<code>point_size</code>	Numeric point size.
<code>jitter_width</code>	Numeric horizontal jitter width.
<code>violin_fill</code>	Character fill color for violins.
<code>violin_color</code>	Character outline color for violins.
<code>violin_alpha</code>	Numeric violin transparency.
<code>smooth_color</code>	Character color for linear trend line.
<code>base_size</code>	Numeric base font size for theme.

Value

A ggplot object.

plot_pca_biplot	<i>PCA biplot for selected feature columns</i>
-----------------	--

Description

Runs principal component analysis (PCA) on selected feature columns after numeric coercion and z-score normalization (\code{center = TRUE, scale. = TRUE}), then draws a biplot with samples as points and feature-loading arrows showing trend directions.

Usage

```
plot_pca_biplot(
  df,
  include_columns,
  color_by,
  color_map = NULL,
  point_size = 2.2,
  point_alpha = 0.9,
  arrow_color = "grey30",
  arrow_alpha = 0.85,
  arrow_scale = 1,
  show_feature_labels = TRUE,
  feature_label_size = 3.5,
  base_size = 12
)
```

Arguments

df	A data.frame containing features and metadata columns.
include_columns	Character vector of column names to include as PCA features.
color_by	Character scalar column name used to color sample points.
color_map	Optional named character vector for sample colors.
point_size	Numeric point size.
point_alpha	Numeric point alpha.
arrow_color	Character arrow/label color for feature loadings.
arrow_alpha	Numeric alpha for loading arrows.
arrow_scale	Numeric multiplier to scale loading-arrow length.
show_feature_labels	Logical; if TRUE, labels loading arrows.
feature_label_size	Numeric feature-label text size.
base_size	Numeric base font size.

Value

A ggplot object. The fitted PCA model is attached as `\codeattr(plot, "pca_model")`, plus score/loading tables in `\codeattr(plot, "pca_scores")` and `\codeattr(plot, "pca_loadings")`.

show, fdObj-method	<i>Display a Summary of an fdObj Object</i>
--------------------	---

Description

The ‘show()’ method provides a concise overview of an fdObj object. It is automatically called when an fdObj is printed in the console (e.g., typing its name). The summary includes the number of registered samples, imported raw curves, successfully transformed approach/retract curves, and metadata column names.

Usage

```
## S4 method for signature 'fdObj'
show(object)
```

Arguments

object An fdObj object.

Details

This method is defined for the S4 class fdObj and is executed automatically when the object is shown in the console. It provides a quick overview of data completeness and analysis progress without modifying the object.

Value

Prints a formatted summary of the object in the console.

Examples

```
## Not run:
folder <- system.file("extdata", package = "curvana")
test <- createFdObjFromFolder(folder)
test # calling 'show()' implicitly

## End(Not run)
```

transform_a_curve	<i>Transform a Single AFM Curve into Separation Distance and Force</i>
-------------------	--

Description

Converts raw AFM deflection and piezo data into calibrated separation distance and force. The zero-separation position is estimated from the sensitivity calibration segment.

Usage

```
transform_a_curve(
  x,
  y,
  baseline,
  sensitivity,
  spring_constant,
  senscal_seg_x,
  senscal_seg_y
)
```

Arguments

x	Numeric vector. Piezo position (distance) in nm.
y	Numeric vector. Deflection signal in volts.
baseline	Numeric. The deflection baseline (in volts).
sensitivity	Numeric. Sensitivity in V/nm.
spring_constant	Numeric. Spring constant in nN/nm.
senscal_seg_x	Numeric vector. X-values of the sensitivity calibration segment.
senscal_seg_y	Numeric vector. Y-values of the sensitivity calibration segment (in volts).

Value

A data.frame with:

separation_distance_nm Tip-sample separation (nm).

force_nN Force (nN).

If any of the input is NULL, an empty dataframe will be returned

transform_curves	<i>Transform All Curves in an fdObj into Separation Distance and Force</i>
------------------	--

Description

Applies ‘transform_a_curve()’ to all raw curves in an fdObj object. Automatically computes sensitivity and baseline values if they are missing by calling ‘analyze_sensitivity()’ and ‘analyze_baseline()’, passing through any user-supplied arguments that match those functions’ parameters.

Usage

```
transform_curves(
  fdObj,
  spring_constant,
  useCurve = c("approach", "retract"),
  threads = 1,
  ...
)
```

Arguments

fdObj	An object of class fdObj.
spring_constant	Numeric or character. If numeric, a constant spring constant (nN/nm) applied to all curves and written to fdObj@metadata\$spring_constant. If character, the name of a column in fdObj@metadata that contains per-curve spring constants.
useCurve	Character; one of c("approach", "retract").
threads	Integer. Number of parallel threads to use (default = 1).
...	Additional arguments passed to ‘analyze_sensitivity()’ or ‘analyze_baseline()’ if they are invoked automatically.

Value

An updated fdObj with transformed curves stored in the corresponding slot.

Index

`analyze_a_curve_adhesive_force`, [2](#)
`analyze_a_curve_area`, [3](#)
`analyze_a_curve_interaction_distance`,
[4](#)
`analyze_a_curve_noise`, [5](#)
`analyze_baseline`, [6](#)
`analyze_curves_adhesive_force`, [7](#)
`analyze_curves_all_analytical_metrics`,
[8](#)
`analyze_curves_energy`, [9](#)
`analyze_curves_interaction_distance`,
[11](#)
`analyze_curves_noise`, [12](#)
`analyze_sensitivity`, [13](#)
`area_trapezoid`, [14](#)
`area_triangle`, [14](#)

`calc_sensitivity`, [15](#)
`combineFdObj`, [16](#)
`createFdObjFromFolder`, [16](#)
`crossing_x0`, [17](#)

`extract`, [18](#)

`fdObj-class`, [18](#)
`find_baseline`, [19](#)

`plot_a_curve_metrics`, [22](#)
`plot_all_curve_metrics`, [20](#)
`plot_complex_heatmap`, [24](#)
`plot_deflection_curves`, [25](#)
`plot_fd_curves`, [26](#)
`plot_force_curve_energy`, [27](#)
`plot_metric_violin`, [28](#)
`plot_pca_biplot`, [30](#)

`show`, `fdObj-method`, [31](#)

`transform_a_curve`, [32](#)
`transform_curves`, [33](#)